

Enterprise Knowledge Assistant – System Design Document

1. Overview

The Enterprise Knowledge Assistant is an AI-powered Retrieval-Augmented Generation (RAG) system that enables users to interact with enterprise documents using natural language. Instead of relying solely on a Large Language Model (LLM), the system retrieves relevant information from uploaded documents, ensuring responses are grounded, explainable, and supported by source citations.

The application combines hybrid retrieval techniques, semantic search, keyword search, re-ranking, conversational memory, and local LLM inference to deliver accurate responses while maintaining user privacy.

2. Technology Stack

Layer	Technology	Purpose
Programming Language	Python 3.12	Core backend application development and AI pipeline implementation.
Backend Framework	Flask	Provides REST APIs, routing, document upload, and web application services.
Frontend	HTML5, CSS3, JavaScript	Implements the interactive user interface, chat window, document management, and responsive design.
Large Language Model (LLM)	Ollama (Llama 3.2)	Generates natural language responses locally using retrieved document context while maintaining data privacy.
Embedding Model	Sentence Transformers (all-MiniLM-L6-v2)	Converts document chunks and user queries into dense vector representations for semantic search.
Vector Database	ChromaDB	Stores vector embeddings and performs efficient similarity search for document retrieval.
Keyword Retrieval	BM25	Performs lexical keyword-based retrieval to complement semantic vector search.

Layer	Technology	Purpose
Hybrid Search	Reciprocal Rank Fusion (RRF)	Combines rankings from BM25 and vector retrieval to improve retrieval robustness and accuracy.
Re-ranking Model	CrossEncoder (MS MARCO MiniLM)	Re-ranks retrieved passages using deep semantic matching to improve context quality before answer generation.
Document Loader	PyMuPDF	Extracts text and metadata from uploaded PDF documents.
Text Chunking	LangChain Recursive Character Text Splitter	Splits documents into overlapping chunks while preserving semantic context for retrieval.
Conversation Memory	Custom Memory Module	Stores recent user interactions to support contextual multi-turn conversations.
Evaluation Framework	Custom Evaluation Module	Measures retrieval precision, recall, answer similarity, latency, confidence, and overall system performance.
Version Control	Git & GitHub	Source code management, collaboration, and project version tracking.
Development Environment	Visual Studio Code	Primary IDE used for application development, debugging, and testing.
Deployment (Current)	Localhost (Flask Development Server)	Executes the application in a local development environment using Ollama for local inference.
Future Deployment	Docker, Kubernetes (Planned)	Containerized deployment and orchestration for scalable enterprise environments.

3. Evaluation Approach

The Enterprise Knowledge Assistant was evaluated using a custom evaluation framework designed to assess both retrieval effectiveness and response quality. A benchmark dataset consisting of representative enterprise questions was created from multiple policy documents, including HR, IT Security, Employee Handbook, and Travel Policy PDFs.

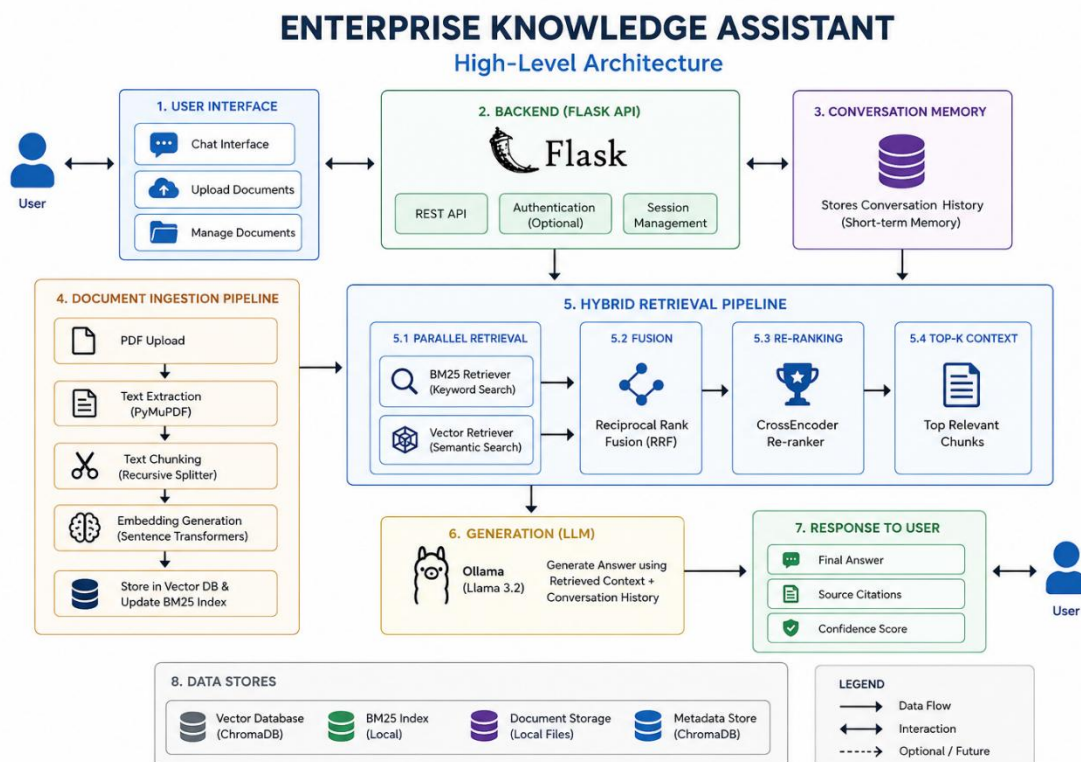
For each test query, the system retrieves relevant document chunks using the hybrid retrieval pipeline (BM25 + Vector Search), re-ranks the retrieved passages using a CrossEncoder model, and generates a response using the Ollama Llama 3.2 model.

The following evaluation metrics were used:

- **Answer Similarity:** Measures how closely the generated answer matches the expected answer.
- **Retrieval Precision:** Measures the proportion of retrieved documents that are relevant to the query.
- **Retrieval Recall:** Measures whether the system successfully retrieves the expected source documents.
- **Response Latency:** Records the total time required to retrieve relevant context and generate the final response.
- **Confidence Score:** Indicates the model's confidence in the generated response based on the retrieval pipeline.
- **Source Verification:** Ensures that generated answers are supported by the retrieved document citations.

The evaluation framework enables quantitative comparison of different retrieval strategies and supports future benchmarking as the document collection grows.

4. High-Level Architecture



5. Data Flow

Document Indexing

1. User uploads a PDF document.
2. The document loader extracts textual content.
3. Documents are divided into overlapping chunks using recursive text splitting.
4. Sentence Transformer embeddings are generated for each chunk.
5. Chunks and embeddings are stored in ChromaDB.
6. BM25 indexes are refreshed to support keyword retrieval.

Query Processing

1. User submits a question through the web interface.
2. Conversation history is added to provide context.
3. Hybrid retrieval performs:
 - BM25 keyword search
 - Dense vector similarity search
4. Reciprocal Rank Fusion combines both result sets.
5. CrossEncoder reranks the retrieved chunks based on semantic relevance.
6. The highest-ranked chunks are passed to the Ollama LLM.
7. The LLM generates a grounded response using only the retrieved context.
8. The system returns:
 - Generated answer
 - Confidence score
 - Supporting document citations

6. Component Explanation

Flask Web Application

Provides the user interface and REST API endpoints for chatting, document upload, and document management.

Document Loader

Extracts text from PDF files using PyMuPDF while preserving metadata such as filename and page number.

Chunking Module

Splits documents into semantically meaningful overlapping chunks, improving retrieval quality while preserving context.

Embedding Generator

Converts text chunks into dense vector representations using Sentence Transformers for semantic search.

ChromaDB

Stores vector embeddings and supports efficient nearest-neighbor retrieval.

BM25 Retriever

Provides traditional keyword-based retrieval, improving performance on exact terminology and abbreviations.

Hybrid Retriever

Combines BM25 and vector search to balance lexical matching with semantic understanding.

Reciprocal Rank Fusion (RRF)

Merges rankings from BM25 and vector retrieval without requiring manual score normalization.

CrossEncoder Re-ranker

Re-evaluates retrieved passages using deep semantic matching and selects the most relevant context before generation.

Conversation Memory

Maintains recent dialogue history, enabling contextual follow-up questions and multi-turn conversations.

Ollama LLM

Generates natural language responses locally using the Llama 3.2 model, ensuring data privacy and eliminating dependency on external APIs.

Evaluation Module

Measures retrieval precision, recall, latency, confidence, and answer similarity to assess overall system performance.

8. Limitations and Future Improvements

Although the proposed system provides accurate document-grounded question answering, several limitations remain.

Current Limitations

- Conversation memory is maintained only during the current application session and is not persisted across restarts.
- Confidence scores are heuristic-based and do not directly reflect retrieval or generation uncertainty.
- The current implementation processes PDF documents only; support for Word, Excel, PowerPoint, and web content is not yet available.
- Document indexing is performed locally and may become slower as the number of documents increases.
- The application currently serves a single-user environment and does not include authentication or user-specific document collections.
- Response generation is synchronous, resulting in higher perceived latency for large language model inference.

Future Improvements

- Implement **query rewriting** to improve retrieval for conversational follow-up questions.
- Introduce **streaming responses** using Server-Sent Events (SSE) or WebSockets for a ChatGPT-like user experience.
- Replace heuristic confidence estimation with retrieval-score or CrossEncoder-based confidence scoring.
- Support additional document formats such as DOCX, PPTX, TXT, HTML, and Markdown.
- Add user authentication and role-based document access for enterprise deployment.
- Deploy the application using Docker and Kubernetes for scalable cloud deployment.
- Integrate advanced RAG evaluation frameworks such as **RAGAS** or **DeepEval** to measure faithfulness, answer relevance, context precision, and hallucination rate.
- Introduce document versioning, incremental synchronization, and automated indexing pipelines for enterprise knowledge management.
- Optimize inference through response caching, embedding caching, and asynchronous background indexing to reduce latency.

9. Conclusion

The Enterprise Knowledge Assistant adopts a modern Retrieval-Augmented Generation architecture that combines hybrid retrieval, semantic reranking, conversational memory,

and local LLM inference. This design improves factual accuracy, reduces hallucinations, provides transparent source attribution, and enables scalable document-based question answering suitable for enterprise knowledge management systems.